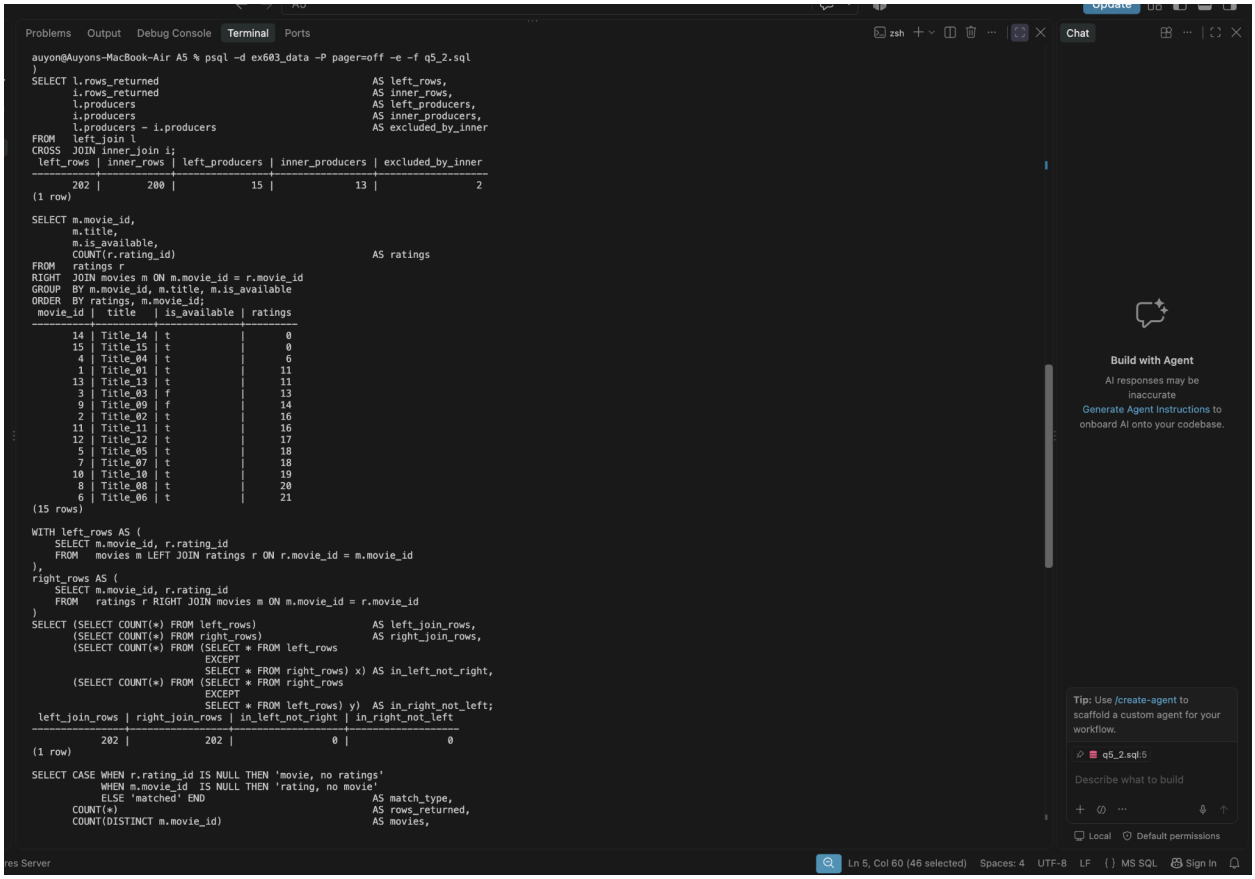
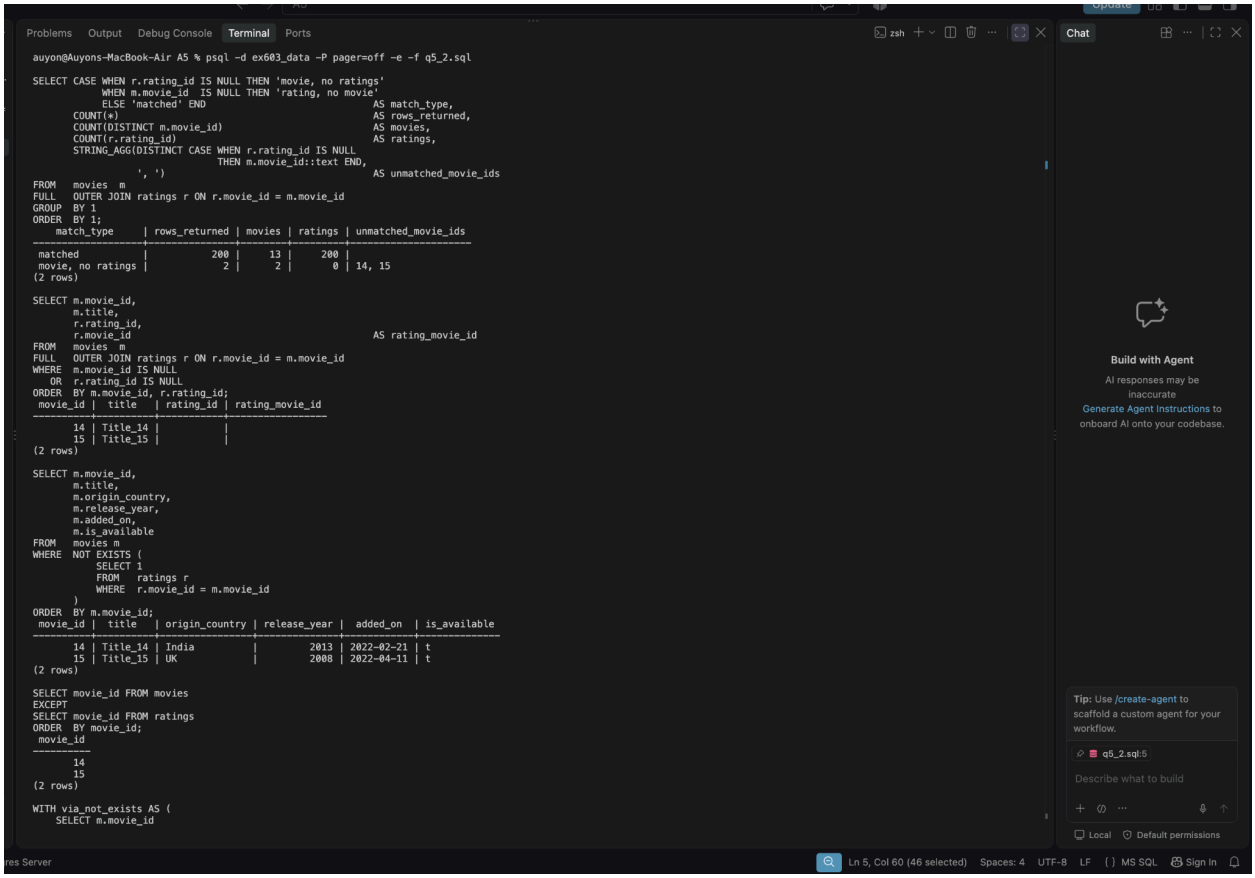



```
auyon@Auyons-MacBook-Air A5 % psql -d ex603_data -P pager=off -e -f q5_2.sql
)
SELECT l.rows_returned                                  AS left_rows,
       i.rows_returned                                  AS inner_rows,
       l.producers                                      AS left_producers,
       i.producers                                      AS inner_producers,
       l.producers - i.producers                        AS excluded_by_inner
FROM   left_join l
CROSS  JOIN inner_join i;
 left_rows | inner_rows | left_producers | inner_producers | excluded_by_inner
-----------+------------+----------------+-----------------+-------------------
       202 |        200 |             15 |              13 |                 2
(1 row)

SELECT m.movie_id,
       m.title,
       m.is_available,
       COUNT(r.rating_id)                               AS ratings
FROM   ratings r
RIGHT  JOIN movies m ON m.movie_id = r.movie_id
GROUP  BY m.movie_id, m.title, m.is_available
ORDER  BY ratings, m.movie_id;
 movie_id |  title   | is_available | ratings
----------+----------+--------------+---------
       14 | Title_14 | t            |       0
       15 | Title_15 | t            |       0
        4 | Title_04 | t            |       6
        1 | Title_01 | t            |      11
       13 | Title_13 | t            |      11
        3 | Title_03 | f            |      13
        9 | Title_09 | f            |      14
        2 | Title_02 | t            |      16
       11 | Title_11 | t            |      16
       12 | Title_12 | t            |      17
        5 | Title_05 | t            |      18
        7 | Title_07 | t            |      18
       10 | Title_10 | t            |      19
        8 | Title_08 | t            |      20
        6 | Title_06 | t            |      21
(15 rows)

WITH left_rows AS (
    SELECT m.movie_id, r.rating_id
    FROM   movies m LEFT JOIN ratings r ON r.movie_id = m.movie_id
),
right_rows AS (
    SELECT m.movie_id, r.rating_id
    FROM   ratings r RIGHT JOIN movies m ON m.movie_id = r.movie_id
)
SELECT (SELECT COUNT(*) FROM left_rows)                 AS left_join_rows,
       (SELECT COUNT(*) FROM right_rows)                AS right_join_rows,
       (SELECT COUNT(*) FROM (SELECT * FROM left_rows
                              EXCEPT
                              SELECT * FROM right_rows) x) AS in_left_not_right,
       (SELECT COUNT(*) FROM (SELECT * FROM right_rows
                              EXCEPT
                              SELECT * FROM left_rows) y)  AS in_right_not_left;
 left_join_rows | right_join_rows | in_left_not_right | in_right_not_left
----------------+-----------------+-------------------+-------------------
            202 |             202 |                 0 |                 0
(1 row)

SELECT CASE WHEN r.rating_id IS NULL THEN 'movie, no ratings'
            WHEN m.movie_id  IS NULL THEN 'rating, no movie'
            ELSE 'matched' END                          AS match_type,
       COUNT(*)                                         AS rows_returned,
       COUNT(DISTINCT m.movie_id)                       AS movies,
```


Problems
Output
Debug Console
Terminal
Ports
zsh
auyon@Auyons-MacBook-Air A5 % psql -d ex603_data -P pager=off -e -f q5_2.sql
SELECT CASE WHEN r.rating_id IS NULL THEN 'movie, no ratings'
WHEN m.movie_id IS NULL THEN 'rating, no movie'
ELSE 'matched' END AS match_type,
COUNT(*) AS rows_returned,
COUNT(DISTINCT m.movie_id) AS movies,
COUNT(r.rating_id) AS ratings,
STRING_AGG(DISTINCT CASE WHEN r.rating_id IS NULL
THEN m.movie_id::text END,
', ') AS unmatched_movie_ids
FROM movies m
FULL OUTER JOIN ratings r ON r.movie_id = m.movie_id
GROUP BY 1
ORDER BY 1;
match_type | rows_returned | movies | ratings | unmatched_movie_ids
matched | 200 | 13 | 200 |
movie, no ratings | 2 | 2 | 0 | 14, 15
(2 rows)
SELECT m.movie_id,
m.title,
r.rating_id,
r.movie_id AS rating_movie_id
FROM movies m
FULL OUTER JOIN ratings r ON r.movie_id = m.movie_id
WHERE m.movie_id IS NULL
OR r.rating_id IS NULL
ORDER BY m.movie_id, r.rating_id;
movie_id | title | rating_id | rating_movie_id
14 | Title_14 | |
15 | Title_15 | |
(2 rows)
SELECT m.movie_id,
m.title,
m.origin_country,
m.release_year,
m.added_on,
m.is_available
FROM movies m
WHERE NOT EXISTS (
SELECT 1
FROM ratings r
WHERE r.movie_id = m.movie_id
)
ORDER BY m.movie_id;
movie_id | title | origin_country | release_year | added_on | is_available
14 | Title_14 | India | 2013 | 2022-02-21 | t
15 | Title_15 | UK | 2008 | 2022-04-11 | t
(2 rows)
SELECT movie_id FROM movies
EXCEPT
SELECT movie_id FROM ratings
ORDER BY movie_id;
movie_id
14
15
(2 rows)
WITH via_not_exists AS (
SELECT m.movie_id
Chat
Build with Agent
AI responses may be inaccurate
Generate Agent Instructions to onboard AI onto your codebase.
Tip: Use /create-agent to scaffold a custom agent for your workflow.
q5_2.sql:5
Describe what to build
Local
Default permissions
Ln 5, Col 60 (46 selected)
Spaces: 4
UTF-8
LF
MS SQL
Sign In